Qwen3-TTS

Source https://github.com/vllm-project/vllm-omni/tree/main/examples/online_serving/qwen3_tts.

🛠️ Installation

Please refer to README.md

Supported Models

Model	Task Type	Description
`Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice`	CustomVoice	Predefined speaker voices with optional style control
`Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign`	VoiceDesign	Natural language voice style description
`Qwen/Qwen3-TTS-12Hz-1.7B-Base`	Base	Voice cloning from reference audio
`Qwen/Qwen3-TTS-12Hz-0.6B-CustomVoice`	CustomVoice	Smaller/faster variant
`Qwen/Qwen3-TTS-12Hz-0.6B-Base`	Base	Smaller/faster variant for voice cloning

Gradio Demo

Two interactive Gradio demos are available, both supporting all 3 task type

stable

Demo	File	Transport	Streaming Quality
Standard	`gradio_demo.py`	HTTP chunked	May have small gaps between chunks
FastRTC	`gradio_fastrtc_demo.py`	WebRTC	Gapless streaming (requires `pip install fastrtc`)

```
# Option 1: Launch server + Standard Gradio together
./run_gradio_demo.sh                                # CustomVoice (default)
./run_gradio_demo.sh --task-type VoiceDesign        # VoiceDesign
./run_gradio_demo.sh --task-type Base               # Voice cloning

# Option 2: If server is already running
python gradio_demo.py --api-base http://localhost:8000

# Option 3: FastRTC demo (gapless streaming)
pip install fastrtc
python gradio_fastrtc_demo.py --api-base http://localhost:8000
```

Then open http://localhost:7860 in your browser.

Run examples (Qwen3-TTS)

Launch the Server

The default stage config is located at `vllm_omni/model_executor/stage_configs/qwen3_tts.yaml`. For other platforms (e.g., NPU), refer to `vllm_omni/platforms/npu/stage_configs/qwen3_tts.yaml`.

stable

```
# CustomVoice model (predefined speakers)
vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice \
    --omni \
    --port 8091

# VoiceDesign model
vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign \
    --omni \
    --port 8091

# Base model (voice cloning)
vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-Base \
    --omni \
    --port 8091
```

If you have custom stage configs file, launch the server with command below

```
vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice \
    --stage-configs-path /path/to/stage_configs_file \
    --omni \
    --port 8091
```

Alternatively, use the convenience script:

```
./run_server.sh                 # Default: CustomVoice model
./run_server.sh CustomVoice     # CustomVoice model
./run_server.sh VoiceDesign     # VoiceDesign model
./run_server.sh Base            # Base (voice clone) model
```

Send TTS Request

Get into the example folder

```
cd examples/online_serving/qwen3_tts
```

Send request via python

```
# CustomVoice: Use predefined speaker
python openai_speech_client.py \
    --model Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice \
    --text "你好，我是通义千问" \
    --voice vivian \
    --language Chinese

# CustomVoice with style instruction
python openai_speech_client.py \
    --model Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice \
```

stable

```
    --text "今天天气真好" \
    --voice ryan \
    --instructions "用开心的语气说"

# VoiceDesign: Describe the voice style
python openai_speech_client.py \
    --model Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign \
    --task-type VoiceDesign \
    --text "哥哥，你回来啦" \
    --instructions "体现撒娇稚嫩的萝莉女声，音调偏高"

# Base: Voice cloning
python openai_speech_client.py \
    --model Qwen/Qwen3-TTS-12Hz-1.7B-Base \
    --task-type Base \
    --text "Hello, this is a cloned voice" \
    --ref-audio /path/to/reference.wav \
    --ref-text "Original transcript of the reference audio"
```

The Python client supports the following command-line arguments:

- `--api-base`: API base URL (default: `http://localhost:8091`)
- `--model` (or `-m`): Model name/path (default: `Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice`)
- `--task-type` (or `-t`): TTS task type. Options: `CustomVoice`, `VoiceDesign`, `Base`
- `--text`: Text to synthesize (required)
- `--voice`: Speaker/voice name (default: `vivian`). Options: `vivian`, `ryan`, `aiden`, etc.
- `--language`: Language. Options: `Auto`, `Chinese`, `English`, `Japanese`, `Korean`, `German`, `French`, `Russian`, `Portuguese`, `Spanish`, `Italian`
- `--instructions`: Voice style/emotion instructions
- `--ref-audio`: Reference audio file path or URL for voice cloning (Base task). Local paths are automatically base64-encoded by the client before sending to the server.
- `--ref-text`: Reference audio transcript for voice cloning (Base task).
- `--response-format`: Audio output format (default: `wav`). Options: `wav`, `mp3`, `flac`, `pcm`, `aac`, `opus`
- `--output` (or `-o`): Output audio file path (default: `tts_output.wav`)

Send request via curl

```
# Simple TTS request
curl -X POST http://localhost:8091/v1/audio/speech \
    -H "Content-Type: application/json" \
    -d '{
        "input": "Hello, how are you?",
```

stable

```
        "voice": "vivian",
        "language": "English"
    }' --output output.wav

# With style instruction
curl -X POST http://localhost:8091/v1/audio/speech \
    -H "Content-Type: application/json" \
    -d '{
        "input": "I am so excited!",
        "voice": "vivian",
        "instructions": "Speak with great enthusiasm"
    }' --output excited.wav

# List available voices in CustomVoice models
curl http://localhost:8091/v1/audio/voices
```

Using OpenAI SDK

```
from openai import OpenAI

client = OpenAI(base_url="http://localhost:8091/v1", api_key="none")

response = client.audio.speech.create(
    model="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
    voice="vivian",
    input="Hello, how are you?",
)

response.stream_to_file("output.wav")
```

Using Python httpx

```
import httpx

response = httpx.post(
    "http://localhost:8091/v1/audio/speech",
    json={
        "input": "Hello, how are you?",
        "voice": "vivian",
        "language": "English",
    },
    timeout=300.0,
)

with open("output.wav", "wb") as f:
    f.write(response.content)
```

stable

FAQ

If you encounter error about backend of librosa, try to install ffmpeg with command below.

```
sudo apt update
sudo apt install ffmpeg
```

API Reference

Voices Endpoint

GET /v1/audio/voices

List all available voices/speakers from the loaded model, including both built-in model voices and uploaded custom voices.

Response Example:

```
{
  "voices": ["vivian", "ryan", "custom_voice_1"],
  "uploaded_voices": [
    {
      "name": "custom_voice_1",
      "consent": "user_consent_id",
      "created_at": 1738660000,
      "file_size": 1024000,
      "mime_type": "audio/wav"
    }
  ]
}
```

POST /v1/audio/voices

Upload a new voice sample for voice cloning in Base task TTS requests.

Form Parameters: - `audio_sample` (required): Audio file (max 10MB, supported formats: wav, mp3, flac, ogg, aac, webm, mp4) - `consent` (required): Consent recording ID - `name` (required): Name for the new voice - `ref_text` (optional): Transcript of the audio. Enables in-context voice cloning (higher quality).

Response Example:

stable

```
{
  "success": true,
  "voice": {
```

```
    "name": "custom_voice_1",
    "consent": "user_consent_id",
    "created_at": 1738660000,
    "mime_type": "audio/wav",
    "file_size": 1024000
  }
}
```

Usage Example:

```
curl -X POST http://localhost:8000/v1/audio/voices \
  -F "audio_sample=@/path/to/voice_sample.wav" \
  -F "consent=user_consent_id" \
  -F "name=custom_voice_1" \
  -F "ref_text=The exact transcript of the audio sample."
```

Endpoint

```
POST /v1/audio/speech
Content-Type: application/json
```

This endpoint follows the OpenAI Audio Speech API format with additional Qwen3-TTS parameters.

Request Body

```
{
    "input": "Text to synthesize",
    "voice": "vivian",
    "response_format": "wav",
    "task_type": "CustomVoice",
    "language": "Auto",
    "instructions": "Optional style instructions",
    "ref_audio":  "HTTP URL, base64 data URL, or file:// URI for voice cloning",
    "ref_text": "Reference audio transcript",
    "x_vector_only_mode": false,
    "max_new_tokens": 2048
}
```

> **Note:** The `model` field is optional when serving a single model, as the server already knows which model is loaded.

stable

Response

Returns binary audio data with appropriate `Content-Type` header (e.g., `audio/wav`).

Parameters

OpenAI Standard Parameters

Parameter	Type	Default	Description
`input`	string	**required**	Text to synthesize
`model`	string	server's model	Model to use (optional, should match server if specified)
`voice`	string	"vivian"	Speaker name (e.g., vivian, ryan, aiden)
`response_format`	string	"wav"	Audio format: wav, mp3, flac, pcm, aac, opus
`speed`	float	1.0	Playback speed (0.25-4.0, not supported with `stream=true`)

vLLM-Omni Extension Parameters

Parameter	Type	Default	Description
`task_type`	string	"CustomVoice"	Task: CustomVoice, VoiceDesign, or Base
`language`	string	"Auto"	Language (see supported languages below)
`instructions`	string	""	Voice style/emotion instructions
`max_new_tokens`	int	2048	Maximum tokens to generate
`initial_codec_chunk_frames`	int	null	Per-request init for TTFA tuning. When null, IC is

stable

Parameter	Type	Default	Description
			computed dynamically based on server load.
`stream`	bool	false	Stream raw PCM chunks as they are decoded (requires `response_format="pcm"`)

Supported languages: Auto, Chinese, English, Japanese, Korean, German, French, Russian, Portuguese, Spanish, Italian

Voice Clone Parameters (Base task)

Parameter	Type	Required	Description
`ref_audio`	string	**Yes**	Reference audio (HTTP URL, base64 data URL, or `file://` URI with `--allowed-local-media-path`)
`ref_text`	string	No	Transcript of reference audio (for ICL mode)
`x_vector_only_mode`	bool	No	Use speaker embedding only (no ICL)

Streaming

Set `stream=true` with `response_format="pcm"` to receive raw PCM audio chunks as they are decoded (one chunk per Code2Wav window, default 25 frames; configurable in the stage config):

```
curl -X POST http://localhost:8091/v1/audio/speech \
    -H "Content-Type: application/json" \
    -d '{
        "input": "Hello, how are you?",
        "voice": "vivian",
        "language": "English",
        "stream": true,
        "response_format": "pcm"
```

stable

```
    }' --no-buffer | play -t raw -r 24000 -e signed -b 16 -c 1 -
```

Constraints: - `stream=true` requires `response_format="pcm"` (raw 16-bit signed PCM, 24 kHz mono). - `speed` adjustment is not supported when streaming. - Requires the server stage config to have `async_chunk: true` (default in `qwen3_tts.yaml`).

Streaming Text Input (WebSocket)

The `/v1/audio/speech/stream` WebSocket endpoint accepts text incrementally, buffers and splits it at sentence boundaries, and generates audio per sentence.

When `stream_audio=true`, each sentence is emitted as `audio.start`, one or more binary PCM frames, and `audio.done`.

Quick Start

```
python streaming_speech_client.py \
    --text "Hello world. How are you? I am fine."

python streaming_speech_client.py \
    --text "Hello world. How are you? I am fine." \
    --simulate-stt --stt-delay 0.1
```

WebSocket Protocol

Client -> Server:

```
{"type": "session.config", "voice": "Vivian", "task_type": "CustomVoice",
"language": "Auto", "split_granularity": "sentence", "stream_audio": true,
"response_format": "pcm"}
{"type": "input.text", "text": "Hello, how are you? "}
{"type": "input.done"}
```

Server -> Client:

```
{"type": "audio.start", "sentence_index": 0, "sentence_text": "Hello, how are
you?", "format": "pcm", "sample_rate": 24000}
// binary PCM frame(s)
{"type": "audio.done", "sentence_index": 0, "total_bytes": 96000, "error": false}
{"type": "session.done", "total_sentences": 1}
```

stable

Limitations

- **Single request**: Batch processing is not yet optimized for online serving.

Troubleshooting

1. **TTS model did not produce audio output**: Ensure you're using the correct model variant for your task type (CustomVoice task → CustomVoice model, etc.)
2. **Connection refused**: Make sure the server is running on the correct port
3. **Out of memory**: Use smaller model variant (`Qwen/Qwen3-TTS-12Hz-0.6B-CustomVoice`) or reduce `--gpu-memory-utilization`
4. **Unsupported speaker**: Use `/v1/audio/voices` to list available voices for the loaded model
5. **Voice clone fails**: Ensure you're using the Base model variant for voice cloning

Example materials

stable

batch_speech_client.py

```
"""Batch speech client for Qwen3-TTS via /v1/audio/speech/batch endpoint.

This script demonstrates how to synthesize multiple texts in a single request.
A particularly useful scenario is voice cloning: set ref_audio once at the
batch level and generate many utterances in the cloned voice without repeating
the reference for each item.

Start the server (with batch-optimized config for best throughput):

    vllm serve Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice \
        --stage-configs-path
vllm_omni/model_executor/stage_configs/qwen3_tts_batch.yaml \
        --trust-remote-code

Examples:
    # Batch with a predefined voice
    python batch_speech_client.py \
        --texts "Hello, how are you?" "Goodbye, see you later!"

    # Voice cloning: one ref_audio, many outputs
    python batch_speech_client.py \
        --task-type Base \
        --ref-audio /path/to/reference.wav \
        --ref-text "Transcript of the reference audio" \
        --texts "First cloned sentence." "Second cloned sentence." \
               "Third cloned sentence."
"""

import argparse
import base64
import os

import httpx

DEFAULT_API_BASE = "http://localhost:8091"
DEFAULT_API_KEY = "EMPTY"


def encode_audio_to_base64(audio_path: str) -> str:
    """Encode a local audio file to a base64 data URL."""
    if not os.path.exists(audio_path):
        raise FileNotFoundError(f"Audio file not found: {audio_path}")

    ext = os.path.splitext(audio_path)[1].lower()
    mime_map = {".wav": "audio/wav", ".mp3": "audio/mpeg", ".flac": "audio/flac",
".ogg": "audio/ogg"}
    mime_type = mime_map.get(ext, "audio/wav")

    with open(audio_path, "rb") as f:
        audio_b64 = base64.b64encode(f.read()).decode("utf-8")
    return f"data:{mime_type};base64,{audio_b64}"
```

stable

```python
def run_batch(args) -> None:
    """Send a batch TTS request and save each result to a file."""
    items = [{"input": text} for text in args.texts]

    payload: dict = {
        "items": items,
        "response_format": args.response_format,
    }
    if args.voice:
        payload["voice"] = args.voice
    if args.language:
        payload["language"] = args.language
    if args.task_type:
        payload["task_type"] = args.task_type
    if args.instructions:
        payload["instructions"] = args.instructions
    if args.max_new_tokens:
        payload["max_new_tokens"] = args.max_new_tokens

    # Voice cloning parameters (shared across all items)
    if args.ref_audio:
        if args.ref_audio.startswith(("http://", "https://")):
            payload["ref_audio"] = args.ref_audio
        else:
            payload["ref_audio"] = encode_audio_to_base64(args.ref_audio)
    if args.ref_text:
        payload["ref_text"] = args.ref_text

    print(f"Sending batch of {len(items)} item(s) to {args.api_base}")
    if args.ref_audio:
        print("Voice cloning mode — ref_audio applied to all items")

    url = f"{args.api_base}/v1/audio/speech/batch"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {args.api_key}",
    }

    with httpx.Client(timeout=300.0) as client:
        response = client.post(url, json=payload, headers=headers)

    if response.status_code != 200:
        print(f"Error {response.status_code}: {response.text}")
        return

    data = response.json()
    print(f"Total: {data['total']}  Succeeded: {data['succeeded']}  Failed:
{data['failed']}")

    os.makedirs(args.output_dir, exist_ok=True)
    for result in data["results"]:
        idx = result["index"]
        if result["status"] == "success":
            audio_bytes = base64.b64decode(result["audio_data"])
            out_path = os.path.join(args.output_dir, f"batch_{idx}.
{args.response_format}")
            with open(out_path, "wb") as f:
```

stable

```
                f.write(audio_bytes)
            print(f"  [{idx}] saved {len(audio_bytes)} bytes -> {out_path}")
        else:
            print(f"  [{idx}] FAILED: {result['error']}")


def parse_args():
    parser = argparse.ArgumentParser(
        description="Batch speech client for /v1/audio/speech/batch",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__,
    )

    parser.add_argument("--api-base", default=DEFAULT_API_BASE, help="API base
URL")
    parser.add_argument("--api-key", default=DEFAULT_API_KEY, help="API key")

    # Texts to synthesize
    parser.add_argument(
        "--texts",
        nargs="+",
        required=True,
        help="One or more texts to synthesize",
    )

    # Shared voice settings
    parser.add_argument("--voice", default="vivian", help="Speaker name (default:
vivian)")
    parser.add_argument("--language", default=None, help="Language: Auto, Chinese,
English, etc.")
    parser.add_argument("--instructions", default=None, help="Voice style/emotion
instructions")
    parser.add_argument(
        "--task-type",
        default=None,
        choices=["CustomVoice", "VoiceDesign", "Base"],
        help="TTS task type (default: CustomVoice)",
    )

    # Voice cloning (Base task)
    parser.add_argument("--ref-audio", default=None, help="Reference audio path or
URL for voice cloning")
    parser.add_argument("--ref-text", default=None, help="Reference audio
transcript for voice cloning")

    # Generation
    parser.add_argument("--max-new-tokens", type=int, default=None, help="Max new
tokens per item")
    parser.add_argument(
        "--response-format",
        default="wav",
        choices=["wav", "mp3", "flac", "pcm", "aac", "opus"],
        help="Audio format (default: wav)",
    )
    parser.add_argument("--output-dir", "-o", default="batch_output", help="Output
directory (default: batch_output)")
```

stable

```
    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    run_batch(args)
```

stable

gradio_demo.py

```python
"""Gradio demo for Qwen3-TTS with gapless streaming audio playback.

Uses a custom AudioWorklet-based player for gap-free streaming,
inspired by github.com/KoljaB/RealtimeVoiceChat. Audio is streamed
from the vLLM server through a same-origin proxy and played via the
Web Audio API's AudioWorklet, which maintains a FIFO buffer queue
and plays samples at the audio clock rate — eliminating inter-chunk
gaps inherent in Gradio's built-in streaming audio component.

Also supports non-streaming mode (full audio download) via gr.Audio.

Supports all 3 task types:
  - CustomVoice: Predefined speaker with optional style instructions
  - VoiceDesign: Natural language voice description
  - Base: Voice cloning from reference audio (upload or URL)

Usage:
    # Start the vLLM server first (see run_server.sh), then:
    python gradio_demo.py --api-base http://localhost:8000
"""

import argparse
import io
import json
import logging

import gradio as gr
import httpx
import numpy as np
import soundfile as sf
from fastapi import FastAPI, Request
from fastapi.responses import Response, StreamingResponse
from tts_common import (
    PCM_SAMPLE_RATE,
    SUPPORTED_LANGUAGES,
    TASK_TYPES,
    add_common_args,
    build_payload,
    fetch_voices,
)

logger = logging.getLogger(__name__)

# ── AudioWorklet processor (loaded in browser via Blob URL) ──────────
WORKLET_JS = r"""
class TTSPlaybackProcessor extends AudioWorkletProcessor {
    constructor() {
        super();
        this.queue = [];
        this.buf = null;
        this.pos = 0;
        this.playing = false;
        this.played = 0;
```

stable

```
        this.port.onmessage = (e) => {
            if (e.data && e.data.type === 'clear') {
                this.queue = []; this.buf = null; this.pos = 0; this.played = 0;
                if (this.playing) { this.playing = false;
this.port.postMessage({type:'stopped'}); }
                return;
            }
            this.queue.push(e.data);
        };
    }
    process(inputs, outputs) {
        const out = outputs[0][0];
        for (let i = 0; i < out.length; i++) {
            if (!this.buf || this.pos >= this.buf.length) {
                if (this.queue.length > 0) {
                    this.buf = this.queue.shift(); this.pos = 0;
                } else {
                    for (let j = i; j < out.length; j++) out[j] = 0;
                    if (this.playing) { this.playing = false;
this.port.postMessage({type:'stopped', played:this.played}); }
                    return true;
                }
            }
            out[i] = this.buf[this.pos++] / 32768;
            this.played++;
        }
        if (!this.playing) { this.playing = true;
this.port.postMessage({type:'started'}); }
        return true;
    }
}
registerProcessor('tts-playback-processor', TTSPlaybackProcessor);
"""

# ── Player HTML (container with metric cards) ──────────────────────────
PLAYER_HTML = """
<div id="tts-player">
  <div style="display:flex; align-items:center; gap:10px;">
    <div id="tts-status-dot" style="width:10px;height:10px;border-
radius:50%;background:#ccc;flex-shrink:0;"></div>
    <span id="tts-status" style="font-weight:600;font-size:1.05em;">Ready</span>
    <button id="tts-stop-btn" onclick="window.ttsStop()"
      style="display:none; margin-left:auto; padding:5px 16px; border-radius:6px;
border:1px solid #EF5552;
             background:#fff; color:#EF5552; cursor:pointer; font-size:0.85em;
transition:all 0.15s;">Stop</button>
  </div>
  <div id="tts-metrics" style="display:none; grid-template-columns:repeat(4,1fr);
gap:10px; margin-top:12px;">
    <div style="background:#f8f9fa;border-radius:6px;padding:8px 10px;text-
align:center;">
      <div style="font-size:0.7em;text-transform:uppercase;color
spacing:0.5px;margin-bottom:2px;">TTFP</div>
      <div id="tts-m-ttfp" style="font-size:1.2em;font-weight:70
</div>
    </div>
```

stable

```
    <div style="background:#f8f9fa;border-radius:6px;padding:8px 10px;text-
align:center;">
      <div style="font-size:0.7em;text-transform:uppercase;color:#888;letter-
spacing:0.5px;margin-bottom:2px;">RTF</div>
      <div id="tts-m-rtf" style="font-size:1.2em;font-weight:700;color:#333;">–
</div>
    </div>
    <div style="background:#f8f9fa;border-radius:6px;padding:8px 10px;text-
align:center;">
      <div style="font-size:0.7em;text-transform:uppercase;color:#888;letter-
spacing:0.5px;margin-bottom:2px;">Audio</div>
      <div id="tts-m-dur" style="font-size:1.2em;font-weight:700;color:#333;">–
</div>
    </div>
    <div style="background:#f8f9fa;border-radius:6px;padding:8px 10px;text-
align:center;">
      <div style="font-size:0.7em;text-transform:uppercase;color:#888;letter-
spacing:0.5px;margin-bottom:2px;">Speed</div>
      <div id="tts-m-speed" style="font-size:1.2em;font-weight:700;color:#333;">–
</div>
    </div>
  </div>
  <div id="tts-rtf-bar-wrap" style="display:none; background:#e8ecf1; border-
radius:4px; height:20px; overflow:hidden; position:relative; margin-top:10px;">
    <div id="tts-rtf-bar" style="height:100%; border-radius:4px; transition:width
0.3s ease, background 0.3s ease; width:0%;"></div>
    <span id="tts-rtf-label"
style="position:absolute;top:50%;left:50%;transform:translate(-50%,-50%);font-
size:0.75em;font-weight:600;color:#444;"></span>
  </div>
  <div id="tts-elapsed" style="display:none; margin-top:6px; font-size:0.8em;
color:#999; text-align:right;"></div>
</div>
"""


def _build_player_js(sample_rate: int) -> str:
    """Build the JavaScript that powers the AudioWorklet player."""
    return f"""
() => {{
    const SR = {sample_rate};
    const WC = {json.dumps(WORKLET_JS)};
    let ctx = null, node = null, abort = null, gen = false, st = {{}};

    async function init() {{
        if (ctx) return;
        ctx = new AudioContext({{ sampleRate: SR }});
        const b = new Blob([WC], {{ type: 'application/javascript' }});
        const u = URL.createObjectURL(b);
        await ctx.audioWorklet.addModule(u);
        URL.revokeObjectURL(u);
        node = new AudioWorkletNode(ctx, 'tts-playback-processor
        node.connect(ctx.destination);
        node.port.onmessage = (e) => {{
            if (e.data.type === 'started') setStatus('Playing...', '#64dd17');
            else if (e.data.type === 'stopped' && !gen) {{
                setStatus('Done', '#64dd17'); showStats(true);
```

stable

```
                const btn = document.getElementById('tts-stop-btn');
                if (btn) btn.style.display = 'none';
            }}
        }};
    }}

    function setStatus(text, color) {{
        const s = document.getElementById('tts-status');
        const d = document.getElementById('tts-status-dot');
        if (s) s.textContent = text;
        if (d) d.style.background = color || '#ccc';
    }}

    function showStats(fin) {{
        if (!st.t0) return;
        const elapsed = (fin && st.streamEnd ? (st.streamEnd - st.t0) :
(performance.now() - st.t0)) / 1000;
        const dur = st.samples / SR;
        const mTtfp = document.getElementById('tts-m-ttfp');
        const mRtf = document.getElementById('tts-m-rtf');
        const mDur = document.getElementById('tts-m-dur');
        const mSpeed = document.getElementById('tts-m-speed');
        const bar = document.getElementById('tts-rtf-bar');
        const barLabel = document.getElementById('tts-rtf-label');
        const elapsedEl = document.getElementById('tts-elapsed');

        if (mTtfp && st.ttfp != null) mTtfp.textContent = st.ttfp.toFixed(0) +
'ms';
        if (mDur) mDur.textContent = dur.toFixed(1) + 's';

        if (dur > 0 && elapsed > 0) {{
            const rtf = elapsed / dur;
            const speed = 1 / rtf;
            if (mRtf) {{
                mRtf.textContent = rtf.toFixed(2) + 'x';
                mRtf.style.color = rtf < 1 ? '#64dd17' : rtf < 1.5 ? '#e8a317' :
'#EF5552';
            }}
            if (mSpeed) {{
                mSpeed.textContent = speed.toFixed(1) + 'x';
                mSpeed.style.color = speed > 1 ? '#64dd17' : speed > 0.7 ?
'#e8a317' : '#EF5552';
            }}
            if (bar) {{
                const pct = Math.min(speed / 3 * 100, 100);
                bar.style.width = pct + '%';
                bar.style.background = speed > 1 ? 'linear-
gradient(90deg,#4A90D9,#64dd17)' : speed > 0.7 ? 'linear-
gradient(90deg,#e8a317,#f0b866)' : 'linear-gradient(90deg,#EF5552,#f87171)';
            }}
            if (barLabel) barLabel.textContent = speed.toFixed(1) + 'x realtime';
        }}
        if (elapsedEl) {{
            elapsedEl.style.display = 'block';
            elapsedEl.textContent = fin ? 'Completed in ' + elapsed.toFixed(1) + 's
(' + st.chunks + ' chunks)' : elapsed.toFixed(1) + 's elapsed  (' + st.chunks + '
chunks)';
```

stable

```
        }}
    }}

    window.ttsStop = function() {{
        if (abort) abort.abort();
        if (node) node.port.postMessage({{ type: 'clear' }});
        gen = false;
        setStatus('Stopped', '#999');
        const btn = document.getElementById('tts-stop-btn');
        if (btn) btn.style.display = 'none';
    }};

    window.ttsGenerate = async function(payload) {{
        try {{ await init(); if (ctx.state === 'suspended') await ctx.resume(); }}
        catch (e) {{ const s = document.getElementById('tts-status'); if (s)
s.textContent = 'Audio init error: ' + e.message; return; }}

        // Abort previous request and clear worklet buffer
        if (abort) abort.abort();
        node.port.postMessage({{ type: 'clear' }});
        // Wait for worklet to process clear before sending new data
        await new Promise(r => setTimeout(r, 50));
        node.port.postMessage({{ type: 'clear' }});

        gen = true;
        st = {{ t0: null, chunks: 0, samples: 0, ttfp: null }};
        setStatus('Connecting...', '#4A90D9');
        const bEl = document.getElementById('tts-stop-btn');
        if (bEl) bEl.style.display = 'inline-block';
        const mp = document.getElementById('tts-metrics');
        if (mp) {{ mp.style.display = 'grid'; ['tts-m-ttfp','tts-m-rtf','tts-m-
dur','tts-m-speed'].forEach(id => {{ const e = document.getElementById(id); if(e)
{{ e.textContent = '–'; e.style.color = '#333'; }} }}); }}
        const bw = document.getElementById('tts-rtf-bar-wrap');
        if (bw) bw.style.display = 'block';
        const bar = document.getElementById('tts-rtf-bar');
        if (bar) bar.style.width = '0%';
        const bl = document.getElementById('tts-rtf-label');
        if (bl) bl.textContent = '';
        const ee = document.getElementById('tts-elapsed');
        if (ee) {{ ee.style.display = 'none'; ee.textContent = ''; }}
        abort = new AbortController();

        try {{
            console.log('fetch payload:', JSON.stringify({{input:
payload.input?.slice(0,30), task: payload.task_type, has_ref: !!payload.ref_audio,
stream: payload.stream}}));
            st.t0 = performance.now();
            const r = await fetch('/proxy/v1/audio/speech', {{
                method: 'POST',
                headers: {{ 'Content-Type': 'application/json' }},
                body: JSON.stringify(payload),
                signal: abort.signal,
            }});
            if (!r.ok) {{ const t = await r.text(); throw new Error('Server ' +
r.status + ': ' + t.slice(0, 200)); }}
            setStatus('Streaming...', '#4A90D9');
```

stable

```
            const reader = r.body.getReader();
            let left = new Uint8Array(0);
            while (true) {{
                const {{ done, value }} = await reader.read();
                if (done) break;
                let raw;
                if (left.length > 0) {{
                    raw = new Uint8Array(left.length + value.length);
                    raw.set(left); raw.set(value, left.length);
                }} else {{ raw = value; }}
                const usable = raw.length - (raw.length % 2);
                left = usable < raw.length ? raw.slice(usable) : new Uint8Array(0);
                if (usable > 0) {{
                    const ab = new ArrayBuffer(usable);
                    new Uint8Array(ab).set(raw.subarray(0, usable));
                    const pcm = new Int16Array(ab);
                    node.port.postMessage(pcm);
                    st.chunks++;
                    st.samples += pcm.length;
                    if (st.ttfp == null) st.ttfp = performance.now() - st.t0;
                    showStats(false);
                }}
            }}
        }} catch (e) {{
            if (e.name !== 'AbortError') {{
                setStatus('Error: ' + e.message, '#EF5552');
                console.error('TTS error:', e);
            }}
        }} finally {{
            // Freeze RTF at stream-end time (before playback finishes)
            st.streamEnd = performance.now();
            showStats(true);
            gen = false;
            if (st.samples > 0) setStatus('Finishing playback...', '#64dd17');
            else {{
                setStatus('No audio received', '#999');
                if (bEl) bEl.style.display = 'none';
            }}
        }}
    }};
}}
"""


def generate_speech(
    api_base: str,
    text: str,
    task_type: str,
    voice: str,
    language: str,
    instructions: str,
    ref_audio: tuple | None,
    ref_audio_url: str,
    ref_text: str,
    x_vector_only: bool,
    response_format: str,
```

stable

```
    speed: float,
):
    """Non-streaming: call /v1/audio/speech and return complete audio."""
    payload = build_payload(
        text,
        task_type,
        voice,
        language,
        instructions,
        ref_audio,
        ref_audio_url,
        ref_text,
        x_vector_only,
        response_format,
        speed,
        stream=False,
    )
    try:
        with httpx.Client(timeout=300.0) as client:
            resp = client.post(
                f"{api_base}/v1/audio/speech",
                json=payload,
                headers={"Content-Type": "application/json", "Authorization":
"Bearer EMPTY"},
            )
    except httpx.TimeoutException:
        raise gr.Error("Request timed out. The server may be busy.")
    except httpx.ConnectError:
        raise gr.Error(f"Cannot connect to server at {api_base}.")

    if resp.status_code != 200:
        raise gr.Error(f"Server error ({resp.status_code}): {resp.text}")

    content_type = resp.headers.get("content-type", "")
    if "application/json" in content_type:
        try:
            raise gr.Error(f"Server error: {resp.json()}")
        except ValueError:
            pass

    try:
        if response_format == "pcm":
            audio_np = np.frombuffer(resp.content,
dtype=np.int16).astype(np.float32) / 32767.0
            return (PCM_SAMPLE_RATE, audio_np)
        audio_np, sample_rate = sf.read(io.BytesIO(resp.content))
        if audio_np.ndim > 1:
            audio_np = audio_np[:, 0]
        return (sample_rate, audio_np.astype(np.float32))
    except Exception as e:
        raise gr.Error(f"Failed to decode audio: {e}")


def create_app(api_base: str):
    """Create the FastAPI app with streaming proxy + Gradio UI."""
    fastapi_app = FastAPI()
```

stable

```
    # Server-side payload store (avoids sending large base64 through Gradio
textbox)
    _pending_payloads: dict[str, dict] = {}

    # ── Streaming proxy (same-origin, no CORS issues) ──────────────
    @fastapi_app.post("/proxy/v1/audio/speech")
    async def proxy_speech(request: Request):
        body = await request.json()
        # Check if this is a request ID referencing a stored payload
        req_id = body.get("_req_id")
        if req_id and req_id in _pending_payloads:
            body = _pending_payloads.pop(req_id)
        # Pass ref_audio URL directly to vLLM server (it handles URL resolution).
        # Pre-downloading and re-encoding adds ~2-3s to TTFP for large files.
        logger.info(
            "Proxy request: %s",
            {k: (f"<{len(str(v))} chars>" if k == "ref_audio" else v) for k, v in
body.items()},
        )
        try:
            client = httpx.AsyncClient(timeout=300)
            resp = await client.send(
                client.build_request(
                    "POST",
                    f"{api_base}/v1/audio/speech",
                    json=body,
                    headers={"Authorization": "Bearer EMPTY", "Content-Type":
"application/json"},
                ),
                stream=True,
            )
        except Exception as exc:
            logger.exception("Proxy connection error")
            await client.aclose()
            return Response(content=str(exc), status_code=502)

        if resp.status_code != 200:
            content = await resp.aread()
            logger.error("Proxy upstream error %d: %s", resp.status_code,
content[:200])
            await resp.aclose()
            await client.aclose()
            return Response(content=content, status_code=resp.status_code)

        async def relay():
            total = 0
            try:
                async for chunk in resp.aiter_bytes():
                    total += len(chunk)
                    yield chunk
            except Exception:
                logger.exception("Proxy relay error after %d byte
            finally:
                logger.info("Proxy relay done: %d bytes", total)
                await resp.aclose()
                await client.aclose()
```

stable

```
        return StreamingResponse(relay(), media_type="application/octet-stream")

    # ── Gradio UI ──────────────────────────────────────────────
    voices = fetch_voices(api_base)

    css = """
    #generate-btn button { width: 100%; }
    #streaming-player { border: 1px solid var(--border-color-primary) !important;
border-radius: var(--block-radius) !important; padding: var(--block-padding)
!important; }
    """

    theme = gr.themes.Default(
        primary_hue=gr.themes.Color(
            c50="#f0f5ff",
            c100="#dce6f9",
            c200="#b8cef3",
            c300="#8eb2eb",
            c400="#6496e0",
            c500="#4A90D9",
            c600="#3a7bc8",
            c700="#2d66b0",
            c800="#1f4f8f",
            c900="#163a6e",
            c950="#0e2650",
        ),
    )

    with gr.Blocks(
        css=css,
        title="Qwen3-TTS Demo",
        js=_build_player_js(PCM_SAMPLE_RATE),
        theme=theme,
    ) as demo:
        gr.HTML(f"""
        <div style="display:flex; align-items:center; gap:16px; margin-
bottom:8px;">
          <img src="https://raw.githubusercontent.com/vllm-project/vllm-
omni/main/docs/source/logos/vllm-omni-logo.png"
               alt="vLLM-Omni" style="height:42px;">
          <div>
            <h1 style="margin:0; font-size:1.5em;">Qwen3-TTS Streaming Demo</h1>
            <span style="font-size:0.85em; color:#666;">
              Served by <a href="https://github.com/vllm-project/vllm-omni"
target="_blank"
              style="color:#4A90D9; text-decoration:none; font-weight:600;">vLLM-
Omni</a>
               &middot;  <code style="background:#eef2f7; padding:2px
6px; border-radius:4px; font-size:0.9em;">{api_base}</code>
            </span>
          </div>
        </div>
        """)

        with gr.Row():
            with gr.Column(scale=3):
                text_input = gr.Textbox(
```

stable

```
                    label="Text to Synthesize",
                    placeholder="Enter text here, e.g., Hello, how are you?",
                    lines=4,
                )
                with gr.Row():
                    task_type = gr.Radio(
                        choices=TASK_TYPES,
                        value="CustomVoice",
                        label="Task Type",
                        scale=2,
                    )
                    language = gr.Dropdown(
                        choices=SUPPORTED_LANGUAGES,
                        value="Auto",
                        label="Language",
                        scale=1,
                    )
                voice = gr.Dropdown(
                    choices=voices,
                    value=voices[0] if voices else None,
                    label="speaker",
                    visible=True,
                    allow_custom_value=True,
                )
                instructions = gr.Textbox(
                    label="Instructions",
                    placeholder="e.g., Speak with excitement / A warm, friendly
female voice",
                    lines=2,
                    visible=True,
                    info="Optional style/emotion instructions",
                )
                with gr.Column(visible=False) as ref_group:
                    ref_audio = gr.Audio(
                        label="Reference Audio (upload for voice cloning)",
                        type="numpy",
                        sources=["upload", "microphone"],
                    )
                    ref_audio_url = gr.Textbox(
                        label="Reference Audio URL",
                        placeholder="https://example.com/reference.wav",
                        lines=1,
                    )
                    ref_text = gr.Textbox(
                        label="Reference Audio Transcript",
                        placeholder="Transcript of reference audio (optional,
improves quality)",
                        lines=2,
                    )
                    x_vector_only = gr.Checkbox(
                        label="Use x-vector only",
                        value=False,
                        info="Skip reference transcript, use spea
only",
                    )

                with gr.Row():
```

stable

```
                    response_format = gr.Dropdown(
                        choices=["wav", "mp3", "flac", "pcm", "aac", "opus"],
                        value="pcm",
                        label="Audio Format",
                        scale=1,
                        interactive=False,
                    )
                    speed = gr.Slider(
                        minimum=0.25,
                        maximum=4.0,
                        value=1.0,
                        step=0.05,
                        label="Speed",
                        scale=1,
                        interactive=False,
                    )
                    stream_checkbox = gr.Checkbox(
                        label="Stream (gapless)",
                        value=True,
                        info="AudioWorklet streaming (recommended)",
                        scale=1,
                    )

                with gr.Row():
                    generate_btn = gr.Button(
                        "Generate Speech",
                        variant="primary",
                        size="lg",
                        elem_id="generate-btn",
                        scale=3,
                    )
                    reset_btn = gr.Button(
                        "Reset",
                        variant="secondary",
                        size="lg",
                        scale=1,
                    )

            with gr.Column(scale=2):
                player_html = gr.HTML(
                    value=PLAYER_HTML,
                    visible=True,
                    label="streaming player",
                    elem_id="streaming-player",
                )
                audio_output = gr.Audio(
                    label="generated audio",
                    interactive=False,
                    autoplay=True,
                    visible=False,
                )
                with gr.Column(visible=True) as examples_cv:
                    gr.Examples(
                        examples=[
                            [
                                "Have you ever wondered what it would be like to
travel through time and visit ancient civilizations? The possibilities are endless,
```

stable

```
from witnessing the construction of the pyramids to experiencing the Renaissance
firsthand.",
                                "ryan",
                                "English",
                                "",
                            ],
                            [
                                "其实我真的有发现，我是一个特别善于观察别人情绪的人。比如说在
一个聚会上，我总能第一时间察觉到谁不太开心，然后想办法让大家都能融入到欢乐的氛围中来。",
                                "vivian",
                                "Chinese",
                                "用特别愤怒的语气说",
                            ],
                            [
                                "It was a dark and stormy night when the old
lighthouse keeper heard a knock at the door. He set down his cup of tea, adjusted
his glasses, and walked slowly toward the entrance.",
                                "aiden",
                                "English",
                                "Speak in a mysterious, suspenseful tone",
                            ],
                        ],
                        inputs=[text_input, voice, language, instructions],
                        label="examples",
                    )
                with gr.Column(visible=False) as examples_vd:
                    gr.Examples(
                        examples=[
                            [
                                "It's in the top drawer... wait, it's empty? No
way, that's impossible! I'm sure I put it there! Someone must have moved it while I
was gone.",
                                "English",
                                "Speak in an incredulous tone, but with a hint of
panic beginning to creep into your voice.",
                            ],
                            [
                                "哥哥，你回来啦，人家等了你好久好久了，要抱抱！你去哪里了呀，
都不跟人家说一声，人家好担心你哦。",
                                "Chinese",
                                "体现撒娇稚嫩的萝莉女声，音调偏高且起伏明显，营造出黏人、做作
又刻意卖萌的听觉效果。",
                            ],
                        ],
                        inputs=[text_input, language, instructions],
                        label="examples",
                    )
                with gr.Column(visible=False) as examples_base:
                    gr.Examples(
                        examples=[
                            [
                                "Good one. Okay, fine, I'm just g
sock monkey here. Goodbye. I hope you enjoy the rest of your ever
wonderful time.",
                                "English",
                                "https://qianwen-res.oss-cn-
beijing.aliyuncs.com/Qwen3-TTS-Repo/clone_2.wav",
```

stable

```
                                "Okay. Yeah. I resent you. I love you. I respect
you. But you know what? You blew it! And thanks to you.",
                            ],
                            [
                                "其实我真的有发现，我是一个特别善于观察别人情绪的人。比如说在
一个聚会上，我总能第一时间察觉到谁不太开心。",
                                "Chinese",
                                "https://qianwen-res.oss-cn-
beijing.aliyuncs.com/Qwen3-TTS-Repo/clone_2.wav",
                                "Okay. Yeah. I resent you. I love you. I respect
you. But you know what? You blew it! And thanks to you.",
                            ],
                        ],
                        inputs=[text_input, language, ref_audio_url, ref_text],
                        label="examples",
                    )
                gr.HTML("""
                <div style="text-align:center; padding:8px 0; margin-top:4px;">
                  <a href="https://github.com/vllm-project/vllm-omni"
target="_blank">
                    <img src="https://raw.githubusercontent.com/vllm-project/vllm-
omni/main/docs/source/logos/vllm-omni-logo.png"
                         alt="vLLM-Omni" style="height:28px; opacity:0.7;">
                  </a>
                </div>
                """)

        # Hidden textbox to pass payload from Python → JavaScript
        hidden_payload = gr.Textbox(visible=False, elem_id="tts-payload")

        # ── Event wiring ──────────────────────────────────────────
        def on_task_change(tt):
            is_cv = tt == "CustomVoice"
            is_vd = tt == "VoiceDesign"
            is_base = tt == "Base"
            return (
                gr.update(visible=is_cv),  # voice
                gr.update(
                    visible=(is_cv or is_vd),
                    info="Required: describe the voice style" if is_vd else
"Optional style/emotion instructions",
                ),  # instructions
                gr.update(visible=is_base),  # ref_group
                gr.update(visible=is_cv),  # examples_cv
                gr.update(visible=is_vd),  # examples_vd
                gr.update(visible=is_base),  # examples_base
            )

        task_type.change(
            fn=on_task_change,
            inputs=[task_type],
            outputs=[
                voice,
                instructions,
                ref_group,
                examples_cv,
                examples_vd,
```

stable

```
                examples_base,
            ],
        )

        def on_stream_change(stream: bool):
            if stream:
                return (
                    gr.update(value="pcm", interactive=False),
                    gr.update(interactive=False),
                    gr.update(visible=True),  # player
                    gr.update(visible=False),  # audio
                )
            return (
                gr.update(value="wav", interactive=True),
                gr.update(interactive=True),
                gr.update(visible=False),
                gr.update(visible=True),
            )

        stream_checkbox.change(
            fn=on_stream_change,
            inputs=[stream_checkbox],
            outputs=[response_format, speed, player_html, audio_output],
        )

        def on_reset():
            return (
                "",  # text
                None,  # audio_output
                "",  # hidden_payload
                PLAYER_HTML,  # reset player
            )

        reset_btn.click(
            fn=on_reset,
            outputs=[text_input, audio_output, hidden_payload, player_html],
            js="() => { if (window.ttsStop) window.ttsStop(); }",
        )

        all_inputs = [
            text_input,
            task_type,
            voice,
            language,
            instructions,
            ref_audio,
            ref_audio_url,
            ref_text,
            x_vector_only,
            response_format,
            speed,
        ]

        def on_generate(stream_enabled, *args):
            if stream_enabled:
                import time as _time
```

stable

```
                text, task_type_v, voice_v, lang_v, instr_v, ref_a, ref_url, ref_t,
xvec, _fmt, _spd = args
                # For streaming, use URL for ref_audio (base64 is too large
                # for the Gradio textbox → JS → fetch pipeline).
                # If user uploaded audio but no URL, encode and store server-side.
                if ref_a is not None and not (ref_url and ref_url.strip()):
                    req_id = f"req-{int(_time.time() * 1000)}"
                    full_payload = build_payload(
                        text,
                        task_type_v,
                        voice_v,
                        lang_v,
                        instr_v,
                        ref_a,
                        ref_url,
                        ref_t,
                        xvec,
                        stream=True,
                    )
                    _pending_payloads[req_id] = full_payload
                    browser_payload = {"_req_id": req_id, "_nonce":
int(_time.time() * 1000)}
                    return json.dumps(browser_payload), gr.update()

                # URL-only path: payload is small, pass directly to browser
                payload = build_payload(
                    text,
                    task_type_v,
                    voice_v,
                    lang_v,
                    instr_v,
                    None,
                    ref_url,
                    ref_t,
                    xvec,
                    stream=True,
                )
                payload["_nonce"] = int(_time.time() * 1000)
                return json.dumps(payload), gr.update()
            else:
                audio = generate_speech(api_base, *args)
                return "", audio

        generate_btn.click(
            fn=on_generate,
            inputs=[stream_checkbox] + all_inputs,
            outputs=[hidden_payload, audio_output],
        ).then(
            fn=lambda p: p,
            inputs=[hidden_payload],
            outputs=[hidden_payload],
            js="(p) => { if (p && p.trim()) { const d = JSON.pars
d._nonce; window.ttsGenerate(d); } return p; }",
        )

        demo.queue()
```

stable

```
    return gr.mount_gradio_app(fastapi_app, demo, path="/")


def parse_args():
    parser = argparse.ArgumentParser(
        description="Gradio demo for Qwen3-TTS with gapless AudioWorklet
streaming.",
    )
    add_common_args(parser)
    return parser.parse_args()


def main():
    args = parse_args()
    logging.basicConfig(level=logging.INFO)
    print(f"Connecting to vLLM server at: {args.api_base}")

    app = create_app(args.api_base)

    import uvicorn

    uvicorn.run(app, host=args.host, port=args.port)


if __name__ == "__main__":
    main()
```

stable

openai_speech_client.py

```python
"""OpenAI-compatible client for Qwen3-TTS via /v1/audio/speech endpoint.

This script demonstrates how to use the OpenAI-compatible speech API
to generate audio from text using Qwen3-TTS models.

Examples:
    # CustomVoice task (predefined speaker)
    python openai_speech_client.py --text "Hello, how are you?" --voice vivian

    # CustomVoice with emotion instruction
    python openai_speech_client.py --text "I'm so happy!" --voice vivian \
        --instructions "Speak with excitement"

    # VoiceDesign task (voice from description)
    python openai_speech_client.py --text "Hello world" \
        --task-type VoiceDesign \
        --instructions "A warm, friendly female voice"

    # Base task (voice cloning)
    python openai_speech_client.py --text "Hello world" \
        --task-type Base \
        --ref-audio "https://example.com/reference.wav" \
        --ref-text "This is the reference transcript"

    # Base task with pre-computed speaker embedding
    python openai_speech_client.py --text "Hello world" \
        --task-type Base \
        --speaker-embedding embedding.json
"""

import argparse
import base64
import json
import os

import httpx

# Default server configuration
DEFAULT_API_BASE = "http://localhost:8091"
DEFAULT_API_KEY = "EMPTY"


def encode_audio_to_base64(audio_path: str) -> str:
    """Encode a local audio file to base64 data URL."""
    if not os.path.exists(audio_path):
        raise FileNotFoundError(f"Audio file not found: {audio_path}")

    # Detect MIME type from extension
    audio_path_lower = audio_path.lower()
    if audio_path_lower.endswith(".wav"):
        mime_type = "audio/wav"
    elif audio_path_lower.endswith((".mp3", ".mpeg")):
        mime_type = "audio/mpeg"
```

stable

```
    elif audio_path_lower.endswith(".flac"):
        mime_type = "audio/flac"
    elif audio_path_lower.endswith(".ogg"):
        mime_type = "audio/ogg"
    else:
        mime_type = "audio/wav"  # Default

    with open(audio_path, "rb") as f:
        audio_bytes = f.read()
    audio_b64 = base64.b64encode(audio_bytes).decode("utf-8")
    return f"data:{mime_type};base64,{audio_b64}"


def run_tts_generation(args) -> None:
    """Run TTS generation via OpenAI-compatible /v1/audio/speech API."""

    # Build request payload
    payload = {
        "model": args.model,
        "input": args.text,
        "speaker": args.speaker,
        "response_format": args.response_format,
    }

    # Add optional parameters
    if args.instructions:
        payload["instructions"] = args.instructions
    if args.task_type:
        payload["task_type"] = args.task_type
    if args.language:
        payload["language"] = args.language
    if args.max_new_tokens:
        payload["max_new_tokens"] = args.max_new_tokens

    # Voice clone parameters (Base task)
    if args.ref_audio:
        if args.ref_audio.startswith(("http://", "https://")):
            payload["ref_audio"] = args.ref_audio
        elif args.ref_audio.startswith("data:"):
            payload["ref_audio"] = args.ref_audio
        else:
            payload["ref_audio"] = encode_audio_to_base64(args.ref_audio)
    if args.ref_text:
        payload["ref_text"] = args.ref_text
    if args.x_vector_only:
        payload["x_vector_only_mode"] = True
    if args.speaker_embedding:
        with open(args.speaker_embedding) as f:
            payload["speaker_embedding"] = json.load(f)

    print(f"Model: {args.model}")
    print(f"Task type: {args.task_type or 'CustomVoice'}")
    print(f"Text: {args.text}")
    print(f"Speaker: {args.speaker}")
    print("Generating audio...")

    # Make the API call
```

stable

```
    api_url = f"{args.api_base}/v1/audio/speech"
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {args.api_key}",
    }

    with httpx.Client(timeout=300.0) as client:
        response = client.post(api_url, json=payload, headers=headers)

    if response.status_code != 200:
        print(f"Error: {response.status_code}")
        print(response.text)
        return

    # Check for JSON error response (only if content is valid UTF-8 text)
    try:
        text = response.content.decode("utf-8")
        if text.startswith('{"error"'):
            print(f"Error: {text}")
            return
    except UnicodeDecodeError:
        pass  # Binary audio data, not an error

    # Save audio response
    output_path = args.output or "tts_output.wav"
    with open(output_path, "wb") as f:
        f.write(response.content)
    print(f"Audio saved to: {output_path}")


def parse_args():
    """Parse command line arguments."""
    parser = argparse.ArgumentParser(
        description="OpenAI-compatible client for Qwen3-TTS via /v1/audio/speech",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=__doc__,
    )

    # Server configuration
    parser.add_argument(
        "--api-base",
        type=str,
        default=DEFAULT_API_BASE,
        help=f"API base URL (default: {DEFAULT_API_BASE})",
    )
    parser.add_argument(
        "--api-key",
        type=str,
        default=DEFAULT_API_KEY,
        help="API key (default: EMPTY)",
    )
    parser.add_argument(
        "--model",
        "-m",
        type=str,
        default="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice",
        help="Model name/path",
```

stable

```
    )

    # Task configuration
    parser.add_argument(
        "--task-type",
        "-t",
        type=str,
        default=None,
        choices=["CustomVoice", "VoiceDesign", "Base"],
        help="TTS task type (default: CustomVoice)",
    )

    # Input text
    parser.add_argument(
        "--text",
        type=str,
        required=True,
        help="Text to synthesize",
    )

    # Voice/speaker
    parser.add_argument(
        "--speaker",
        type=str,
        default="vivian",
        help="Speaker name (default: vivian). Options: vivian, ryan, aiden, etc.",
    )
    parser.add_argument(
        "--language",
        type=str,
        default=None,
        help="Language: Auto, Chinese, English, etc.",
    )
    parser.add_argument(
        "--instructions",
        type=str,
        default=None,
        help="Voice style/emotion instructions",
    )

    # Base (voice clone) parameters
    parser.add_argument(
        "--ref-audio",
        type=str,
        default=None,
        help="Reference audio file path, URL, or base64 for voice cloning (Base
task)",
    )
    parser.add_argument(
        "--ref-text",
        type=str,
        default=None,
        help="Reference audio transcript for voice cloning (Base
    )
    parser.add_argument(
        "--x-vector-only",
        action="store_true",
```

stable

```
        help="Use x-vector only mode for voice cloning (no ICL)",
    )
    parser.add_argument(
        "--speaker-embedding",
        type=str,
        default=None,
        help="Path to JSON file containing a pre-computed speaker embedding vector
(1024-dim for 0.6B, 2048-dim for 1.7B)",
    )

    # Generation parameters
    parser.add_argument(
        "--max-new-tokens",
        type=int,
        default=None,
        help="Maximum new tokens to generate",
    )

    # Output
    parser.add_argument(
        "--response-format",
        type=str,
        default="wav",
        choices=["wav", "mp3", "flac", "pcm", "aac", "opus"],
        help="Audio output format (default: wav)",
    )
    parser.add_argument(
        "--output",
        "-o",
        type=str,
        default=None,
        help="Output audio file path (default: tts_output.wav)",
    )

    return parser.parse_args()


if __name__ == "__main__":
    args = parse_args()
    run_tts_generation(args)
```

stable

run_gradio_demo.sh

```bash
#!/bin/bash
# Launch both vLLM server and Gradio demo for Qwen3-TTS
#
# Usage:
#   ./run_gradio_demo.sh                                    # Default: CustomVoice
#   ./run_gradio_demo.sh --task-type VoiceDesign            # VoiceDesign model
#   ./run_gradio_demo.sh --task-type Base --gradio-port 7861
#
# Options:
#   --task-type TYPE        Task type: CustomVoice, VoiceDesign, Base (default:
CustomVoice)
#   --server-port PORT      Port for vLLM server (default: 8000)
#   --gradio-port PORT      Port for Gradio demo (default: 7860)
#   --server-host HOST      Host for vLLM server (default: 0.0.0.0)
#   --gradio-ip IP          IP for Gradio demo (default: 127.0.0.1)
#   --share                 Share Gradio demo publicly

set -e

# Default values
TASK_TYPE="CustomVoice"
SERVER_PORT=8000
GRADIO_PORT=7860
SERVER_HOST="0.0.0.0"
GRADIO_IP="127.0.0.1"
GRADIO_SHARE=false

# Parse command line arguments
while [[ $# -gt 0 ]]; do
    case $1 in
        --task-type)
            TASK_TYPE="$2"
            shift 2
            ;;
        --server-port)
            SERVER_PORT="$2"
            shift 2
            ;;
        --gradio-port)
            GRADIO_PORT="$2"
            shift 2
            ;;
        --server-host)
            SERVER_HOST="$2"
            shift 2
            ;;
        --gradio-ip)
            GRADIO_IP="$2"
            shift 2
            ;;
        --share)
            GRADIO_SHARE=true
            shift
```

stable

```
            ;;
        --help)
            echo "Usage: $0 [OPTIONS]"
            echo ""
            echo "Options:"
            echo "  --task-type TYPE        Task type: CustomVoice, VoiceDesign,
Base (default: CustomVoice)"
            echo "  --server-port PORT      Port for vLLM server (default: 8000)"
            echo "  --gradio-port PORT      Port for Gradio demo (default: 7860)"
            echo "  --server-host HOST      Host for vLLM server (default:
0.0.0.0)"
            echo "  --gradio-ip IP          IP for Gradio demo (default:
127.0.0.1)"
            echo "  --share                 Share Gradio demo publicly"
            echo ""
            exit 0
            ;;
        *)
            echo "Unknown option: $1"
            echo "Use --help for usage information"
            exit 1
            ;;
    esac
done

# Map task type to model
case "$TASK_TYPE" in
    CustomVoice)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice"
        ;;
    VoiceDesign)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign"
        ;;
    Base)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-Base"
        ;;
    *)
        echo "Unknown task type: $TASK_TYPE"
        echo "Supported: CustomVoice, VoiceDesign, Base"
        exit 1
        ;;
esac

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
API_BASE="http://localhost:${SERVER_PORT}"

echo "=========================================="
echo "Qwen3-TTS Gradio Demo"
echo "=========================================="
echo "Task Type : $TASK_TYPE"
echo "Model     : $MODEL"
echo "Server    : http://${SERVER_HOST}:${SERVER_PORT}"
echo "Gradio    : http://${GRADIO_IP}:${GRADIO_PORT}"
echo "=========================================="

# Cleanup on exit
cleanup() {
```

stable

```
    echo ""
    echo "Shutting down..."
    if [ -n "$SERVER_PID" ]; then
        echo "Stopping vLLM server (PID: $SERVER_PID)..."
        kill "$SERVER_PID" 2>/dev/null || true
        wait "$SERVER_PID" 2>/dev/null || true
    fi
    if [ -n "$GRADIO_PID" ]; then
        echo "Stopping Gradio demo (PID: $GRADIO_PID)..."
        kill "$GRADIO_PID" 2>/dev/null || true
        wait "$GRADIO_PID" 2>/dev/null || true
    fi
    echo "Cleanup complete"
    exit 0
}
trap cleanup SIGINT SIGTERM

# Start vLLM server
echo ""
echo "Starting vLLM server..."
LOG_FILE="/tmp/vllm_tts_server_${SERVER_PORT}.log"

vllm-omni serve "$MODEL" \
    --stage-configs-path vllm_omni/model_executor/stage_configs/qwen3_tts.yaml \
    --host "$SERVER_HOST" \
    --port "$SERVER_PORT" \
    --gpu-memory-utilization 0.9 \
    --trust-remote-code \
    --omni 2>&1 | tee "$LOG_FILE" &
SERVER_PID=$!

# Wait for server startup
echo ""
echo "Waiting for vLLM server to be ready..."
STARTUP_FLAG="/tmp/vllm_tts_startup_flag_${SERVER_PORT}.tmp"
rm -f "$STARTUP_FLAG"

(
    tail -f "$LOG_FILE" 2>/dev/null | grep -m 1 "Application startup complete" >
/dev/null && touch "$STARTUP_FLAG"
) &
TAIL_PID=$!

MAX_WAIT=300
ELAPSED=0
while [ $ELAPSED -lt $MAX_WAIT ]; do
    if [ -f "$STARTUP_FLAG" ]; then
        kill "$TAIL_PID" 2>/dev/null || true
        wait "$TAIL_PID" 2>/dev/null || true
        echo ""
        echo "vLLM server is ready!"
        break
    fi
    if ! kill -0 "$SERVER_PID" 2>/dev/null; then
        kill "$TAIL_PID" 2>/dev/null || true
        echo ""
        echo "Error: vLLM server failed to start"
```

stable

```
        exit 1
    fi
    sleep 1
    ELAPSED=$((ELAPSED + 1))
done

rm -f "$STARTUP_FLAG"

if [ $ELAPSED -ge $MAX_WAIT ]; then
    kill "$TAIL_PID" 2>/dev/null || true
    echo "Error: Server startup timed out after ${MAX_WAIT}s"
    kill "$SERVER_PID" 2>/dev/null || true
    exit 1
fi

# Start Gradio demo
echo ""
echo "Starting Gradio demo..."
cd "$SCRIPT_DIR"
GRADIO_CMD=("python" "gradio_demo.py" "--api-base" "$API_BASE" "--host"
"$GRADIO_IP" "--port" "$GRADIO_PORT")
if [ "$GRADIO_SHARE" = true ]; then
    GRADIO_CMD+=("--share")
fi

"${GRADIO_CMD[@]}" &
GRADIO_PID=$!

echo ""
echo "========================================="
echo "Both services are running!"
echo "========================================="
echo "vLLM Server : http://${SERVER_HOST}:${SERVER_PORT}"
echo "Gradio Demo : http://${GRADIO_IP}:${GRADIO_PORT}"
echo ""
echo "Press Ctrl+C to stop both services"
echo "========================================="
echo ""

wait $SERVER_PID $GRADIO_PID || true
cleanup
```

stable

run_server.sh

```bash
#!/bin/bash
# Launch vLLM-Omni server for Qwen3-TTS models
#
# Usage:
#   ./run_server.sh                          # Default: CustomVoice model
#   ./run_server.sh CustomVoice              # CustomVoice model
#   ./run_server.sh VoiceDesign              # VoiceDesign model
#   ./run_server.sh Base                     # Base (voice clone) model

set -e

TASK_TYPE="${1:-CustomVoice}"

case "$TASK_TYPE" in
    CustomVoice)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-CustomVoice"
        ;;
    VoiceDesign)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-VoiceDesign"
        ;;
    Base)
        MODEL="Qwen/Qwen3-TTS-12Hz-1.7B-Base"
        ;;
    *)
        echo "Unknown task type: $TASK_TYPE"
        echo "Supported: CustomVoice, VoiceDesign, Base"
        exit 1
        ;;
esac

echo "Starting Qwen3-TTS server with model: $MODEL"

vllm-omni serve "$MODEL" \
    --stage-configs-path vllm_omni/model_executor/stage_configs/qwen3_tts.yaml \
    --host 0.0.0.0 \
    --port 8091 \
    --gpu-memory-utilization 0.9 \
    --trust-remote-code \
    --omni
```

stable

speaker_embedding_interpolation.py

```python
"""Speaker embedding extraction and interpolation for Qwen3-TTS.

Extracts speaker embeddings from reference audio files using the ECAPA-TDNN
speaker encoder from a Qwen3-TTS checkpoint, then interpolates between them
using SLERP and sends the result to the /v1/audio/speech API.

Requirements:
    pip install torch librosa soundfile numpy httpx

Examples:
    # Extract and save an embedding
    python speaker_embedding_interpolation.py extract \
        --model Qwen/Qwen3-TTS-12Hz-1.7B-Base \
        --audio voice_a.wav \
        --output voice_a_embedding.json

    # Interpolate between two embeddings and generate speech
    python speaker_embedding_interpolation.py interpolate \
        --embedding-a voice_a_embedding.json \
        --embedding-b voice_b_embedding.json \
        --ratio 0.5 \
        --text "Hello, this is a blended voice." \
        --output blended.wav

    # Full pipeline: extract from two audio files + interpolate + generate
    python speaker_embedding_interpolation.py pipeline \
        --model Qwen/Qwen3-TTS-12Hz-1.7B-Base \
        --audio-a voice_a.wav \
        --audio-b voice_b.wav \
        --ratios 0.0 0.25 0.5 0.75 1.0 \
        --text "Hello, this is a blended voice." \
        --output-dir ./interpolated/
"""

import argparse
import json
import os
import sys

import httpx
import numpy as np
import torch

DEFAULT_API_BASE = "http://localhost:8000"
DEFAULT_API_KEY = "EMPTY"

# ────────────────────────────────────────────────
# Speaker embedding extraction (offline)
# ────────────────────────────────────────────────


def load_speaker_encoder(model_path: str, device: str = "cpu") -> torch.nn.Module:
    """Load just the ECAPA-TDNN speaker encoder from a Qwen3-TTS checkpoint.
```

stable

```
    This avoids loading the full TTS model by only instantiating the speaker
    encoder sub-module from the checkpoint weights.
    """
    from transformers import AutoConfig

    # Register the config class so AutoConfig can resolve it
    sys.path.insert(0, os.path.dirname(__file__))
    try:
        from vllm_omni.model_executor.models.qwen3_tts.configuration_qwen3_tts
import (
            Qwen3TTSConfig,
        )
        from vllm_omni.model_executor.models.qwen3_tts.qwen3_tts_talker import (
            Qwen3TTSSpeakerEncoder,
        )

        AutoConfig.register("qwen3_tts", Qwen3TTSConfig)
    except ImportError:
        # If running outside the vllm-omni tree, try importing from the HF hub
        config = AutoConfig.from_pretrained(model_path, trust_remote_code=True)
        # Dynamically import from the downloaded model files
        import importlib

        from huggingface_hub import snapshot_download

        model_dir = snapshot_download(model_path)
        spec = importlib.util.spec_from_file_location(
            "modeling_qwen3_tts",
            os.path.join(model_dir, "modeling_qwen3_tts.py"),
        )
        mod = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(mod)
        Qwen3TTSSpeakerEncoder = mod.Qwen3TTSSpeakerEncoder

        spec2 = importlib.util.spec_from_file_location(
            "configuration_qwen3_tts",
            os.path.join(model_dir, "configuration_qwen3_tts.py"),
        )
        mod2 = importlib.util.module_from_spec(spec2)
        spec2.loader.exec_module(mod2)
        Qwen3TTSConfig = mod2.Qwen3TTSConfig
        config_obj = Qwen3TTSConfig.from_pretrained(model_path)
        encoder = Qwen3TTSSpeakerEncoder(config_obj.speaker_encoder_config)
        # Load weights
        _load_speaker_encoder_weights(encoder, model_path)
        encoder = encoder.to(device).eval()
        return encoder

    config = AutoConfig.from_pretrained(model_path, trust_remote_code=True)
    encoder = Qwen3TTSSpeakerEncoder(config.speaker_encoder_config)

    _load_speaker_encoder_weights(encoder, model_path)
    encoder = encoder.to(device).eval()
    return encoder
```

stable

```
def _load_speaker_encoder_weights(encoder: torch.nn.Module, model_path: str) ->
None:
    """Load only the speaker_encoder.* weights from the checkpoint."""
    from huggingface_hub import snapshot_download
    from safetensors.torch import load_file

    model_dir = snapshot_download(model_path)

    prefix = "speaker_encoder."
    state_dict = {}

    # Try safetensors first, then pytorch bin
    safetensor_files = sorted(f for f in os.listdir(model_dir) if
f.endswith(".safetensors"))
    if safetensor_files:
        for fname in safetensor_files:
            shard = load_file(os.path.join(model_dir, fname))
            for k, v in shard.items():
                if k.startswith(prefix):
                    state_dict[k[len(prefix) :]] = v
    else:
        bin_files = sorted(f for f in os.listdir(model_dir) if f.endswith(".bin"))
        for fname in bin_files:
            shard = torch.load(os.path.join(model_dir, fname), map_location="cpu",
weights_only=True)
            for k, v in shard.items():
                if k.startswith(prefix):
                    state_dict[k[len(prefix) :]] = v

    if not state_dict:
        raise RuntimeError(
            f"No speaker_encoder weights found in {model_path}. Make sure this is a
Qwen3-TTS-*-Base checkpoint."
        )

    encoder.load_state_dict(state_dict)


def compute_mel_spectrogram(audio: np.ndarray, sr: int = 24000) -> torch.Tensor:
    """Compute 128-bin mel spectrogram matching Qwen3-TTS's extraction pipeline."""
    import librosa

    # Resample to 24kHz if needed
    if sr != 24000:
        audio = librosa.resample(audio.astype(np.float32), orig_sr=sr,
target_sr=24000)

    y = torch.from_numpy(audio).unsqueeze(0).float()

    from librosa.filters import mel as librosa_mel_fn

    mel_basis = torch.from_numpy(librosa_mel_fn(sr=24000, n_fft=
fmin=0, fmax=12000)).float()

    n_fft = 1024
    hop_size = 256
    win_size = 1024
```

stable

```
    padding = (n_fft - hop_size) // 2
    y = torch.nn.functional.pad(y.unsqueeze(1), (padding, padding),
mode="reflect").squeeze(1)

    hann_window = torch.hann_window(win_size)
    spec = torch.stft(
        y,
        n_fft,
        hop_length=hop_size,
        win_length=win_size,
        window=hann_window,
        center=False,
        return_complex=True,
    )
    spec = torch.abs(spec)
    mel = torch.matmul(mel_basis, spec)
    mel = torch.log(torch.clamp(mel, min=1e-5))
    return mel.transpose(1, 2)  # (1, T, 128)


@torch.inference_mode()
def extract_embedding(encoder: torch.nn.Module, audio_path: str, device: str =
"cpu") -> np.ndarray:
    """Extract a 1024-dim speaker embedding from an audio file."""
    import librosa

    audio, sr = librosa.load(audio_path, sr=None, mono=True)
    mel = compute_mel_spectrogram(audio, sr).to(device)
    embedding = encoder(mel.to(next(encoder.parameters()).dtype))[0]
    return embedding.float().cpu().numpy()


# ────────────────────────────────────────────────
# Interpolation
# ────────────────────────────────────────────────


def slerp(v0: np.ndarray, v1: np.ndarray, t: float) -> np.ndarray:
    """Spherical linear interpolation between two vectors."""
    v0_norm = v0 / (np.linalg.norm(v0) + 1e-8)
    v1_norm = v1 / (np.linalg.norm(v1) + 1e-8)

    dot = np.clip(np.dot(v0_norm, v1_norm), -1.0, 1.0)
    omega = np.arccos(dot)

    if np.abs(omega) < 1e-6:
        # Vectors are nearly parallel, fall back to lerp
        return (1.0 - t) * v0 + t * v1

    sin_omega = np.sin(omega)
    return (np.sin((1.0 - t) * omega) / sin_omega) * v0 + (np.sin(t * omega) /
sin_omega) * v1


# ────────────────────────────────────────────────
# API client
# ────────────────────────────────────────────────
```

stable

```
def generate_speech(
    text: str,
    speaker_embedding: list[float],
    output_path: str,
    api_base: str = DEFAULT_API_BASE,
    api_key: str = DEFAULT_API_KEY,
    model: str = "Qwen/Qwen3-TTS-12Hz-1.7B-Base",
) -> None:
    """Send a speaker_embedding to the TTS API and save the output."""
    payload = {
        "model": model,
        "input": text,
        "task_type": "Base",
        "speaker_embedding": speaker_embedding,
        "response_format": "wav",
    }
    headers = {
        "Content-Type": "application/json",
        "Authorization": f"Bearer {api_key}",
    }
    with httpx.Client(timeout=300.0) as client:
        resp = client.post(f"{api_base}/v1/audio/speech", json=payload,
headers=headers)

    if resp.status_code != 200:
        print(f"Error {resp.status_code}: {resp.text}")
        sys.exit(1)

    with open(output_path, "wb") as f:
        f.write(resp.content)
    print(f"Saved: {output_path}")


# ──────────────────────────────────────────────
# CLI commands
# ──────────────────────────────────────────────


def cmd_extract(args):
    """Extract a speaker embedding from audio and save as JSON."""
    print(f"Loading speaker encoder from {args.model}...")
    encoder = load_speaker_encoder(args.model, device=args.device)

    print(f"Extracting embedding from {args.audio}...")
    emb = extract_embedding(encoder, args.audio, device=args.device)
    print(f"Embedding shape: {emb.shape}")

    output_path = args.output or os.path.splitext(args.audio)[0] +
"_embedding.json"
    with open(output_path, "w") as f:
        json.dump(emb.tolist(), f)
    print(f"Saved embedding to {output_path}")


def cmd_interpolate(args):
```

stable

```
    """Interpolate between two embeddings and generate speech."""
    with open(args.embedding_a) as f:
        emb_a = np.array(json.load(f), dtype=np.float32)
    with open(args.embedding_b) as f:
        emb_b = np.array(json.load(f), dtype=np.float32)

    print(f"Embedding A: {args.embedding_a} (dim={emb_a.shape[0]})")
    print(f"Embedding B: {args.embedding_b} (dim={emb_b.shape[0]})")
    print(f"SLERP ratio: {args.ratio}")

    blended = slerp(emb_a, emb_b, args.ratio)
    output_path = args.output or f"interpolated_t{args.ratio:.2f}.wav"

    print(f"Generating speech: {args.text!r}")
    generate_speech(
        text=args.text,
        speaker_embedding=blended.tolist(),
        output_path=output_path,
        api_base=args.api_base,
        api_key=args.api_key,
        model=args.model,
    )


def cmd_pipeline(args):
    """Full pipeline: extract two embeddings, SLERP at multiple ratios,
generate."""
    print(f"Loading speaker encoder from {args.model}...")
    encoder = load_speaker_encoder(args.model, device=args.device)

    print(f"Extracting embedding A from {args.audio_a}...")
    emb_a = extract_embedding(encoder, args.audio_a, device=args.device)
    print(f"Extracting embedding B from {args.audio_b}...")
    emb_b = extract_embedding(encoder, args.audio_b, device=args.device)

    # Save extracted embeddings
    os.makedirs(args.output_dir, exist_ok=True)
    for label, emb, audio_path in [("a", emb_a, args.audio_a), ("b", emb_b,
args.audio_b)]:
        emb_path = os.path.join(args.output_dir, f"embedding_{label}.json")
        with open(emb_path, "w") as f:
            json.dump(emb.tolist(), f)
        print(f"Saved embedding {label} to {emb_path}")

    # Generate at each ratio
    for t in args.ratios:
        blended = slerp(emb_a, emb_b, t)
        out_path = os.path.join(args.output_dir, f"t{t:.2f}.wav")
        print(f"\n--- SLERP t={t:.2f} ---")
        print(f"Generating: {args.text!r}")
        generate_speech(
            text=args.text,
            speaker_embedding=blended.tolist(),
            output_path=out_path,
            api_base=args.api_base,
            api_key=args.api_key,
            model=args.model,
```

stable

```
        )

    print(f"\nAll outputs saved to {args.output_dir}/")


def main():
    parser = argparse.ArgumentParser(
        description="Speaker embedding extraction and interpolation for Qwen3-TTS",
        formatter_class=argparse.RawDescriptionHelpFormatter,
    )
    parser.add_argument("--api-base", default=DEFAULT_API_BASE, help="TTS API base
URL")
    parser.add_argument("--api-key", default=DEFAULT_API_KEY, help="API key")
    parser.add_argument(
        "--model",
        default="Qwen/Qwen3-TTS-12Hz-1.7B-Base",
        help="Model name (used for both weight loading and API requests)",
    )
    parser.add_argument("--device", default="cpu", help="Device for embedding
extraction (cpu/cuda)")

    sub = parser.add_subparsers(dest="command", required=True)

    # extract
    p_ext = sub.add_parser("extract", help="Extract speaker embedding from audio")
    p_ext.add_argument("--audio", required=True, help="Input audio file")
    p_ext.add_argument("--output", "-o", help="Output JSON path (default:
<audio>_embedding.json)")

    # interpolate
    p_interp = sub.add_parser("interpolate", help="Interpolate between two
embeddings and generate speech")
    p_interp.add_argument("--embedding-a", required=True, help="JSON file with
embedding A")
    p_interp.add_argument("--embedding-b", required=True, help="JSON file with
embedding B")
    p_interp.add_argument("--ratio", "-r", type=float, default=0.5, help="SLERP
ratio (0=A, 1=B)")
    p_interp.add_argument("--text", required=True, help="Text to synthesize")
    p_interp.add_argument("--output", "-o", help="Output wav path")

    # pipeline
    p_pipe = sub.add_parser("pipeline", help="Full pipeline: extract + interpolate
+ generate")
    p_pipe.add_argument("--audio-a", required=True, help="Audio file for voice A")
    p_pipe.add_argument("--audio-b", required=True, help="Audio file for voice B")
    p_pipe.add_argument(
        "--ratios",
        nargs="+",
        type=float,
        default=[0.0, 0.25, 0.5, 0.75, 1.0],
        help="SLERP ratios to generate (default: 0.0 0.25 0.5 0.7
    )
    p_pipe.add_argument("--text", required=True, help="Text to s
    p_pipe.add_argument("--output-dir", default="./interpolated", help="Output
directory")
```

stable

```
    args = parser.parse_args()
    {"extract": cmd_extract, "interpolate": cmd_interpolate, "pipeline":
cmd_pipeline}[args.command](args)


if __name__ == "__main__":
    main()
```

stable

streaming_speech_client.py

```python
"""WebSocket client for streaming text-input TTS.

Connects to the /v1/audio/speech/stream endpoint, sends text incrementally
(simulating real-time STT output), and saves per-sentence audio files.

Usage:
    # Send full text at once
    python streaming_speech_client.py --text "Hello world. How are you? I am fine."

    # Simulate STT: send text word-by-word with delay
    python streaming_speech_client.py \
        --text "Hello world. How are you? I am fine." \
        --simulate-stt --stt-delay 0.1

    # VoiceDesign task
    python streaming_speech_client.py \
        --text "Today is a great day. The weather is nice." \
        --task-type VoiceDesign \
        --instructions "A cheerful young female voice"

    # Base task (voice cloning)
    python streaming_speech_client.py \
        --text "Hello world. How are you?" \
        --task-type Base \
        --ref-audio /path/to/reference.wav \
        --ref-text "Transcript of reference audio"

Requirements:
    pip install websockets
"""

import argparse
import asyncio
import json
import os

try:
    import websockets
except ImportError:
    print("Please install websockets: pip install websockets")
    raise SystemExit(1)


async def stream_tts(
    url: str,
    text: str,
    config: dict,
    output_dir: str,
    simulate_stt: bool = False,
    stt_delay: float = 0.1,
) -> None:
    """Connect to the streaming TTS endpoint and process audio responses."""
    os.makedirs(output_dir, exist_ok=True)
```

stable

```
    async with websockets.connect(url) as ws:
        # 1. Send session config
        config_msg = {"type": "session.config", **config}
        await ws.send(json.dumps(config_msg))
        print(f"Sent session config: {config}")

        # 2. Send text (either all at once or word-by-word)
        async def send_text():
            if simulate_stt:
                words = text.split(" ")
                for i, word in enumerate(words):
                    chunk = word + (" " if i < len(words) - 1 else "")
                    await ws.send(
                        json.dumps(
                            {
                                "type": "input.text",
                                "text": chunk,
                            }
                        )
                    )
                    print(f"  Sent: {chunk!r}")
                    await asyncio.sleep(stt_delay)
            else:
                await ws.send(
                    json.dumps(
                        {
                            "type": "input.text",
                            "text": text,
                        }
                    )
                )
                print(f"Sent full text: {text!r}")

            # 3. Signal end of input
            await ws.send(json.dumps({"type": "input.done"}))
            print("Sent input.done")

        # Run sender and receiver concurrently
        sender_task = asyncio.create_task(send_text())

        response_format = config.get("response_format", "wav")
        current_sentence_index = 0
        current_chunks: list[bytes] = []

        try:
            while True:
                message = await ws.recv()

                if isinstance(message, bytes):
                    current_chunks.append(message)
                    print(f"  Received audio chunk for sentence
{current_sentence_index}: {len(message)} bytes")
                else:
                    # JSON frame
                    msg = json.loads(message)
                    msg_type = msg.get("type")
```

stable

```
                    if msg_type == "audio.start":
                        current_sentence_index = msg["sentence_index"]
                        current_chunks = []
                        print(f"  [sentence {msg['sentence_index']}] Generating:
{msg['sentence_text']!r}")
                    elif msg_type == "audio.done":
                        filename = os.path.join(
                            output_dir,
                            f"sentence_{msg['sentence_index']:03d}.
{response_format}",
                        )
                        with open(filename, "wb") as f:
                            f.write(b"".join(current_chunks))
                        print(
                            f"  [sentence {msg['sentence_index']}] Done"
                            f" bytes={msg.get('total_bytes',
len(b''.join(current_chunks)))}"
                            f" error={msg.get('error', False)}"
                            f" -> {filename}"
                        )
                        current_chunks = []
                    elif msg_type == "session.done":
                        print(f"\nSession complete: {msg['total_sentences']}
sentence(s) generated")
                        break
                    elif msg_type == "error":
                        print(f"  ERROR: {msg['message']}")
                    else:
                        print(f"  Unknown message: {msg}")
        finally:
            sender_task.cancel()
            try:
                await sender_task
            except asyncio.CancelledError:
                pass  # Task cancellation is expected during shutdown

    print(f"\nAudio files saved to: {output_dir}/")


def main():
    parser = argparse.ArgumentParser(description="Streaming text-input TTS client")
    parser.add_argument(
        "--url",
        default="ws://localhost:8000/v1/audio/speech/stream",
        help="WebSocket endpoint URL",
    )
    parser.add_argument(
        "--text",
        required=True,
        help="Text to synthesize",
    )
    parser.add_argument(
        "--output-dir",
        default="streaming_tts_output",
        help="Directory to save audio files (default: streaming_tts_output)",
    )
```

stable

```
    # Session config options
    parser.add_argument("--model", default=None, help="Model name")
    parser.add_argument("--speaker", default="Vivian", help="Speaker name")
    parser.add_argument(
        "--task-type",
        default="CustomVoice",
        choices=["CustomVoice", "VoiceDesign", "Base"],
        help="TTS task type",
    )
    parser.add_argument("--language", default="Auto", help="Language")
    parser.add_argument("--instructions", default=None, help="Voice style
instructions")
    parser.add_argument(
        "--response-format",
        default="wav",
        choices=["wav", "pcm", "flac", "mp3", "aac", "opus"],
        help="Audio format",
    )
    parser.add_argument(
        "--stream-audio",
        action="store_true",
        help="Receive one or more PCM chunks per sentence (requires --response-
format pcm)",
    )
    parser.add_argument("--speed", type=float, default=1.0, help="Playback speed
(0.25-4.0)")
    parser.add_argument("--max-new-tokens", type=int, default=None, help="Max
tokens")

    # Base task options
    parser.add_argument("--ref-audio", default=None, help="Reference audio")
    parser.add_argument("--ref-text", default=None, help="Reference text")
    parser.add_argument(
        "--x-vector-only-mode",
        action="store_true",
        default=False,
        help="Speaker embedding only mode",
    )

    # STT simulation
    parser.add_argument(
        "--simulate-stt",
        action="store_true",
        help="Simulate STT by sending text word-by-word",
    )
    parser.add_argument(
        "--stt-delay",
        type=float,
        default=0.1,
        help="Delay between words in STT simulation (seconds)",
    )

    args = parser.parse_args()

    # Build session config (only include non-None values)
    config = {}
```

stable

```
    for key in [
        "model",
        "speaker",
        "task_type",
        "language",
        "instructions",
        "response_format",
        "speed",
        "max_new_tokens",
        "ref_audio",
        "ref_text",
    ]:
        val = getattr(args, key.replace("-", "_"), None)
        if val is not None:
            config[key] = val
    if args.stream_audio:
        config["stream_audio"] = True
    if args.x_vector_only_mode:
        config["x_vector_only_mode"] = True

    asyncio.run(
        stream_tts(
            url=args.url,
            text=args.text,
            config=config,
            output_dir=args.output_dir,
            simulate_stt=args.simulate_stt,
            stt_delay=args.stt_delay,
        )
    )


if __name__ == "__main__":
    main()
```

stable

tts_common.py

```python
"""Shared constants, helpers, and payload building for Qwen3-TTS Gradio demos."""

import base64
import io

import gradio as gr
import httpx
import numpy as np
import soundfile as sf

SUPPORTED_LANGUAGES = [
    "Auto",
    "Chinese",
    "English",
    "Japanese",
    "Korean",
    "German",
    "French",
    "Russian",
    "Portuguese",
    "Spanish",
    "Italian",
]

TASK_TYPES = ["CustomVoice", "VoiceDesign", "Base"]

PCM_SAMPLE_RATE = 24000

DEFAULT_API_BASE = "http://localhost:8000"


def fetch_voices(api_base: str) -> list[str]:
    """Fetch available voices from the server."""
    try:
        with httpx.Client(timeout=10.0) as client:
            resp = client.get(
                f"{api_base}/v1/audio/voices",
                headers={"Authorization": "Bearer EMPTY"},
            )
        if resp.status_code == 200:
            data = resp.json()
            voices = data.get("voices") or []
            if voices:
                return voices
    except Exception:
        pass
    return ["Vivian", "Ryan"]


def encode_audio_to_base64(audio_data: tuple) -> str:
    """Encode Gradio audio input (sample_rate, numpy_array) to base64 data URL."""
    sample_rate, audio_np = audio_data
```

stable

```
    if audio_np.dtype != np.int16:
        if audio_np.dtype in (np.float32, np.float64):
            audio_np = np.clip(audio_np, -1.0, 1.0)
            audio_np = (audio_np * 32767).astype(np.int16)
        else:
            audio_np = audio_np.astype(np.int16)

    buf = io.BytesIO()
    sf.write(buf, audio_np, sample_rate, format="WAV")
    wav_b64 = base64.b64encode(buf.getvalue()).decode("utf-8")
    return f"data:audio/wav;base64,{wav_b64}"


def build_payload(
    text: str,
    task_type: str,
    voice: str,
    language: str,
    instructions: str,
    ref_audio: tuple | None,
    ref_audio_url: str,
    ref_text: str,
    x_vector_only: bool,
    response_format: str = "pcm",
    speed: float = 1.0,
    stream: bool = True,
) -> dict:
    """Build the /v1/audio/speech request payload.

    Raises gr.Error for invalid input so callers don't need to validate.
    """
    if not text or not text.strip():
        raise gr.Error("Please enter text to synthesize.")

    payload: dict = {
        "input": text.strip(),
        "response_format": "pcm" if stream else response_format,
        "stream": stream,
    }
    if not stream:
        payload["speed"] = speed

    if task_type:
        payload["task_type"] = task_type
    if language:
        payload["language"] = language

    if task_type == "CustomVoice":
        if voice:
            payload["speaker"] = voice
        if instructions and instructions.strip():
            payload["instructions"] = instructions.strip()

    elif task_type == "VoiceDesign":
        if not instructions or not instructions.strip():
            raise gr.Error("VoiceDesign task requires voice style instructions.")
        payload["instructions"] = instructions.strip()
```

stable

```
    elif task_type == "Base":
        ref_audio_url_stripped = ref_audio_url.strip() if ref_audio_url else ""
        if ref_audio_url_stripped:
            payload["ref_audio"] = ref_audio_url_stripped
        elif ref_audio is not None:
            payload["ref_audio"] = encode_audio_to_base64(ref_audio)
        else:
            raise gr.Error("Base (voice clone) task requires reference audio.
Upload a file or provide a URL.")
        if ref_text and ref_text.strip():
            payload["ref_text"] = ref_text.strip()
        if x_vector_only:
            payload["x_vector_only_mode"] = True

    return payload


def on_task_type_change(task_type: str):
    """Update UI visibility based on selected task type."""
    if task_type == "CustomVoice":
        return (
            gr.update(visible=True),  # voice dropdown
            gr.update(visible=True, info="Optional style/emotion instructions"),
            gr.update(visible=False),  # ref_audio
            gr.update(visible=False),  # ref_audio_url
            gr.update(visible=False),  # ref_text
            gr.update(visible=False),  # x_vector_only
        )
    elif task_type == "VoiceDesign":
        return (
            gr.update(visible=False),
            gr.update(visible=True, info="Required: describe the voice style"),
            gr.update(visible=False),
            gr.update(visible=False),
            gr.update(visible=False),
            gr.update(visible=False),
        )
    elif task_type == "Base":
        return (
            gr.update(visible=False),
            gr.update(visible=False),
            gr.update(visible=True),
            gr.update(visible=True),
            gr.update(visible=True),
            gr.update(visible=True),
        )
    return (
        gr.update(visible=True),
        gr.update(visible=True),
        gr.update(visible=False),
        gr.update(visible=False),
        gr.update(visible=False),
        gr.update(visible=False),
    )
```

stable

```python
def stream_pcm_chunks(api_base: str, payload: dict):
    """Stream raw PCM bytes from the server, yielding int16 numpy arrays.

    Handles odd-byte boundaries between network chunks.
    """
    leftover = b""
    with httpx.Client(timeout=300.0) as client:
        with client.stream(
            "POST",
            f"{api_base}/v1/audio/speech",
            json=payload,
            headers={
                "Content-Type": "application/json",
                "Authorization": "Bearer EMPTY",
            },
        ) as resp:
            if resp.status_code != 200:
                resp.read()
                raise gr.Error(f"Server error ({resp.status_code}): {resp.text}")
            for chunk in resp.iter_bytes():
                if not chunk:
                    continue
                raw = leftover + chunk
                usable = len(raw) - (len(raw) % 2)
                leftover = raw[usable:]
                if usable == 0:
                    continue
                yield np.frombuffer(raw[:usable], dtype=np.int16).copy()


def add_common_args(parser):
    """Add CLI arguments shared by both demos."""
    parser.add_argument(
        "--api-base",
        default=DEFAULT_API_BASE,
        help=f"Base URL for the vLLM API server (default: {DEFAULT_API_BASE}).",
    )
    parser.add_argument(
        "--host",
        default="0.0.0.0",
        help="Host/IP for Gradio server (default: 0.0.0.0).",
    )
    parser.add_argument(
        "--port",
        type=int,
        default=7860,
        help="Port for Gradio server (default: 7860).",
    )
    parser.add_argument(
        "--share",
        action="store_true",
        help="Share the Gradio demo publicly.",
    )
    return parser
```

stable